PROYECTO INGENIERÍA DE SOFTWARE 2

ENTREGA NÚMERO 1 - INVESTIGACIÓN Y FUNDAMENTACIÓN

Presentado por:

Santiago Lopez

Oswin Cuaran



Universidad Nacional de Colombia

Facultad de Administración

Administración de Sistemas Informáticos

Segundo semestre del 2026

CONTENIDO

1. Introducción.
2. Objetivos.
3. Caso de estudio real Ficticio: Ferretería Cafetera.
4. Arquitectura monolítica vs Arquitectura de microservicios.
5. Descripción de los sistemas analizados.
6. Argumentación general.
7. ¿Por qué el caso de estudio de Ferretería Cafetera justifica el uso de microservicios?
8. Identificación preliminar de los microservicios.
9. Requerimientos funcionales y No funcionales.
10. Posibles ideas y funcionalidades futuras e innovadoras.
11. Bibliografía .

1. Introducción.

La gestión de inventarios es una de las tareas más importantes para cualquier tipo de negocio que vende productos físicos. El saber que se posee en bodega, que se está agotando, a que proveedor se le adquiere la materia prima y cuánto cuesta cada unidad influye directamente en el número de ventas y en pérdidas. Hoy en día existen muchas herramientas para solucionar esta necesidad, desde ERPs a gran escala como SAP Business One hasta opciones más sencillas como Siigo o alternativas de código abierto como Odoo y ERPNext.

En esta primera entrega se reúne la investigación inicial del proyecto de Ingeniería de Software 2, cuyo propósito es diseñar, implementar y evaluar un sistema de gestión de inventarios basado en una arquitectura de diseño de microservicios. Primero se comparará la arquitectura monolítica con la arquitectura de microservicios para poder entender que se gana y que se pierde al escoger una u otra. Después se analizará cada uno de los cuatro sistemas que ya existen en el mercado en cuanto a arquitectura, manejo de kardex, API y experiencia de usuario. Con la información anterior se definirá el alcance de nuestro sistema y algunas funcionalidades que podrían diferenciarlo.

2. Objetivos.

- Comparar la arquitectura monolítica y la arquitectura de microservicios para identificar sus ventajas y desventajas en un sistema de inventarios.
- Analizar los cuatro sistemas planteados anteriormente con respecto a sus arquitecturas, gestión de kardex, API y experiencia del usuario.
- Definir la empresa real ficticia con la que se implementará la gestión de inventarios.

3. Caso de estudio real Ficticio: Ferretería Cafetera

3.1 Sustentación y presentación de la empresa

Para brindar un alcance concreto y sustentable al sistema de gestión de inventarios que se construirá, se propone como caso de estudio una empresa ficticia pero realista, construida a partir de un perfil típico de pyme distribuidora Manizaleña. Se escoge el enfoque de Ferretería a raíz de que los sistemas de gestión de inventarios principalmente generan una alerta para reabastecimiento como consecuencia de bajo stock sin tener en cuenta otras variables. Brindando un panorama más amplio, una alerta de reabastecimiento o aumento de stock se debe de realizar teniendo en cuenta varios aspectos de su entorno geográfico, ocasiones especiales como black friday o fin de año donde generalmente se requiere más existencias de los productos. En este caso en específico, la demanda de stock necesaria para aliviar las necesidades de los clientes de una ferretería se ve directamente aumentada por la existencia de obras civiles

y obras públicas generadas en su cercanía, y, como consecuencia de lo vivido recientemente por el terremoto, una afectación general de una o varias construcciones al rededor genera una demanda mucho más grande de stock existente.

Nombre de la empresa: Ferretería Cafetera
Enfoque: Distribución mayorista y al detal de materiales de construcción y ferretería.
Tamaño: Pyme en crecimiento.
Ubicación y bodegas: Sede principal y bodega central en Manizales, con dos sucursales y bodegas en Pereira y Armenia.
Canales de venta: Mostrador físico en cada sede y venta mayorista a ferreteros pequeños, constructoras y obras, que hacen pedidos recurrentes por canales virtuales.

3.2 Diagnóstico del problema actual

A día de hoy, Ferretería Cafetera no posee un sistema de gestión de inventarios propiamente dicho. El control se realiza de la siguiente manera:

- El inventario se lleva por documentos tipo kardex de manera manual, actualizando de manera mensual por el encargado de turno de la bodega, por lo que si alguna bodega o sede que carece de un producto necesita realizar una venta por pedido, debe de cerciorarse de la existencia del mismo en otra sede o bodega mediante llamada telefónica o mensaje por canal virtual.
- El proceso de ventas se maneja de manera manual, por lo que al salir un producto no se hace la deducción automática del inventario.
- Las órdenes de compra a proveedores se generan de forma reactiva, sólo se genera una orden cuando el encargado de turno nota la inexistencia o escasez de algún producto, no de forma anticipada por umbrales, histórico de rotación o necesidad geográfica.
- En alta demanda, los vendedores generan una sobrecarga operativa al preguntar por disponibilidad de stock, generando un desbalance de acción entre las otras áreas con respecto a la misma.
- No existe una trazabilidad de un producto, lo que puede generar descuadres de inventario.

4. Arquitectura monolítica vs Arquitectura de microservicios

¿Qué es una arquitectura monolítica?

Una arquitectura monolítica es cuando toda la aplicación diseñada vive en un solo proceso y un solo código de base, tanto la interfaz como la lógica específica y el acceso a los datos están juntos y se despliegan como una sola unidad, generalmente contra una única base de datos.

- **Ventajas principales de la arquitectura monolítica:**
    - Es simple de iniciar, de ensayar y de poner en funcionamiento.
    - Contiene un solo build y un solo servidor.
    - Al tener una sola base de datos, cada acción que se realice ocurre en un mismo commit.
    - Las pruebas de extremo a extremo son directas.
    - Posee una infraestructura más simple, menos herramientas de monitoreo y más fácil de comprender, por lo que genera una reducción en costo operativo.
- **Desventajas principales:**
    - Si un módulo posee mucha carga, se debe de escalar o replicar la aplicación completa.
    - Cualquier cambio por insignificante que sea en uno de sus módulos, puede tumbar todo el servicio del aplicativo.
    - Migrar o actualizar el aplicativo resulta ser más costoso debido a que todo el sistema queda atado a un mismo lenguaje, framework y versión.
    - A medida que el sistema crece, compilar y ensayar el aplicativo se vuelve un proceso cada vez más lento.

¿Qué es una arquitectura de microservicios?

Una arquitectura compuesta esencialmente por microservicios consiste en dividir el mismo aplicativo en varios servicios pequeños e independientes, donde cada uno es responsable de su dominio dentro de la razón de ser del sistema.

- **Ventajas principales:**
    - Se puede actualizar un microservicio en específico sin modificar otro, por lo que se reduce el tiempo necesario para realizar cada modificación.
    - Se ahorran recursos al poder escalar solo un servicio y no toda la aplicación.
    - Si un servicio cae, al estar aislado, el resto del aplicativo puede seguir en funcionamiento.
    - Cada uno de los servicios puede usar el lenguaje o framework más adecuado para cada uno.
    - Cada servicio es pequeño y enfocado a una tarea, por lo que es más fácil de comprender y probar.
- **Desventajas principales:**

- Al tener base de datos por cada servicio, cada transacción que abarca varios servicios no es trivial, por lo que se requiere de sistemas más complejos para asegurar que cada transacción no se corrompa.
- Las pruebas de integración y extremo a extremo son más difíciles de realizar, por lo que aumenta el costo al realizar alguna.
- Su costo de implementación es más alto a comparación de una arquitectura monolítica.

5. Descripción de los sistemas analizados

Arquitectura:

- **SAP business One:**

  ERP empresarial de SAP, diseñada para pymes con procesos complejos en su servicio como manufactura, bodegas, con trazabilidad por lote o número de serie de un producto en específico. Este software licenciado, cliente-servidor sobre SAP HANA/SQL, con una interfaz densa pensada para usuarios de negocios ya entrenados. Utiliza una arquitectura monolítica modular, es decir, se divide en módulos funcionales independientes pero todos comparten el mismo proceso y base de datos.

- **Siigo:**

  Es un sistema cerrado, por lo que no es posible verificar su arquitectura interna con certeza. Al analizar su comportamiento desde la aplicación web sin instalación local, con una sola cuenta que sirve para multiples modulos se puede concluir que es una aplicación SaaS alojada en la nube propia de Siigo, pero no es verificado, limitación explícita reconocida al ser un software de código cerrado.

- **Odoo:**

  Es un ERP de código abierto, donde la idea es que el usuario instale lo que es necesario y va agregando módulos conforme su crecimiento lo requiera. Sigue un patrón módulo-vista-controlador. Aunque cada app o extensión se instala por separado, todas corren en el mismo proceso, comparten base de datos y se comunican a través del ORM del framework, no por red. Es igual que SAP, un monolito modular.

- **ERPNext:**

  Es open source pensado para pequeñas y medianas empresas, con bastante presencia en la industria manufacturera. Funciona sobre Frappe Framework, un framework de python y javascript que ya trae resuelto el acceso a la base de datos, autenticación y una API REST. Su arquitectura es similar a la de Odoo, con módulos aparentemente independientes pero termina siendo un monolito modular.

Kardex:

- **SAP Business One:**

  Ofrece seguimiento de stock en tiempo real en múltiples bodegas, con trazabilidad completa por lote y número de serie e integración con lectores de código de barras para el registro de entrada y salida. La valorización del inventario soporta métodos como FIFO o costos promedio.

- **Siigo:**

  Mantiene un kardex desde el momento en que el producto ingresa al inventario, donde calcula automáticamente el costo promedio y la rentabilidad que ofrece dicho producto. Permite manejar bodegas múltiples con saldo independiente por ubicación y registro de traslado entre ellas.

- **Odoo:**

  Implementa un sistema de doble entrada para el inventario, es decir, cada movimiento de stock queda registrado como una transferencia entre dos ubicaciones, lo que permite reconstruir el recorrido completo de un producto desde que entra hasta el destino final, sin necesidad de un módulo de kardex independiente para esta tarea.

- **ERPNext:**

  Lleva el seguimiento de niveles de inventario y re-abastecimiento, aunque se enfoca principalmente en soportar procesos de manufactura como se menciona anteriormente como consumo de materias, ordenes de producción, en lugar de mostrar una trazabilidad visual como la de Odoo por dar un ejemplo.

API/Documentación:

- SAP Business One expone su propio SDK (DI API / UI API) en lugar de un estándar abierto como REST u OpenAPI, lo que hace que integrarse con él requiera conocer herramientas específicas de SAP.
- Siigo ofrece una API REST propia, pero al ser un producto comercial cerrado, su documentación pública es limitada frente a la de un proyecto open source.
- Odoo pone una API vía XML-RPC/JSON-RPC bien documentada, ampliamente usada por la comunidad para integrar Odoo con sistemas externos.
- ERPNext expone una API REST nativa, generada automáticamente por el Frappe Framework para cada tipo de dato del sistema, con documentación abierta.

En conclusión, los sistemas Open SOurce como Odoo y ERPNext son más transparentes y fáciles de integrar que los otros dos, lo cual es un punto muy relevante para este proyecto ya que se construirá un API Gateway propio y documentado lo que ninguno de los 4 hace de forma completamente abierta y clara.

Experiencia del usuario

- **SAP Business one:**

  Posee una interfaz densa, con demasiados campos y menús interactivos técnicos visibles al mismo tiempo, por lo que se concluye que está construido para usuarios de negocios más capacitados o preparados, no para un usuario nuevo que quiera aprender por su cuenta. Su curva de aprendizaje es alta.
- **Siigo:**

  Al contrario del anterior, Siigo prioriza la simplicidad, donde su interfaz está diseñada para poder ser utilizada por usuarios sin conocimientos técnicos que deseen facturar y controlar su propio inventario sin una curva de aprendizaje alta.
- **Odoo:**

  Posee una interfaz visualmente consistente entre todas las extensiones, con navegación intuitiva por lo que es considerado como uno de los ERP Open Source con mejor experiencia del usuario final del mercado actual.
- **ERPNext:**

  Posee una interfaz menos intuitiva y visualmente atractiva o funcional que la de Odoo, pero cumple su propósito, aunque su curva de aprendizaje es mayor.

En conclusión, hay una relación bastante clara entre el público objetivo y la experiencia del usuario final. Los sistemas pensados para pymes sin personal técnico procuran un diseño de interfaz más intuitiva y simple visualmente, en contraste con las pensadas para empresas con personal técnico capacitado como SAP o para desarrolladores e integradores como ERPNext.

Un punto clave a tener en cuenta es que ningún competidor real del mercado actual usa microservicios puros, todos usan monolitos modulares, ya que resuelven este problema con monolitos maduros donde se prioriza la estabilidad y simplicidad operativa sobre la escalabilidad. Durante este proyecto se explorará la alternativa de microservicios precisamente para practicar y evaluar esta ventaja y desventaja en un caso real a menor escala.

Sistema	Fortaleza principal	Debilidad principal
SAP Business One	Trazabilidad por lote, serie y madurez.	Costos y complejidad para una pyme regional.
Siigo	Simplicidad y cumplimiento DIAN.	Código cerrado y documentación de API limitada.
Odoo	Doble entrada y buena UX.	Funciones avanzadas en la versión de pago.
ERPNext	API REST abierta y sin licencia.	Interfaz menos intuitiva, foco en manufactura.

6. Argumentación general

Uniendo toda la información recolectada durante el proceso de investigación, se muestra que el mercado de gestión de inventarios está dominado por soluciones monolíticas modulares maduras como SAP y Siigo, y por alternativas Open Source con la misma filosofía arquitectónica como Odoo y ERPNext. Ninguno de estos cuatro sistemas analizados adopta microservicios como arquitectura central o principal, lo cual confirma que los microservicios no son un requisito para el dominio del negocio, sino una decisión de ingeniería que se compone y se justifica por otros factores particulares como la necesidad de escalar componentes específicos de forma independiente, la tolerancia a fallos parciales o el objetivo particular pedagógico de estudiar esta arquitectura en un proyecto de la vida real. Esto define el alcance de su sistema propio.

El propósito de este proyecto no es competir en funcionalidad sino realizar una demostración de dominio técnico de una arquitectura distribuida aplicada a un dominio ya conocido por el mercado y enfocado en la gestión de inventarios, tomando como referencia las funcionalidades mínimas viables que comparten entre sí los cuatro sistemas analizados como la gestión de productos, el control de unidades en tienda o stock, gestión de proveedores, gestión de órdenes de compra y reportes básicos de operaciones.

7. ¿Por qué el caso de estudio de Ferretería Cafetera justifica el uso de microservicios?

- Escalabilidad desigual entre módulos: En alta demanda, las consultas de stock de manera manual crecen mucho más que las operaciones de compra y proveedores. Con microservicios se escala solo el servicio de consulta de stock.
- Si alguno de los servicios, por ejemplo el de notificaciones falla, el resto del sistema podrá seguir operando con normalidad sin detener tareas diarias necesarias a realizar.
- Cada uno de los integrantes del grupo podrá encargarse de un servicio en específico sin depender de los demás, por lo que acelera el tiempo de realización necesario para construir el aplicativo.

- Se pueden actualizar los microservicios de manera independiente.

8. Identificación preliminar de los microservicios.

Con base en los dominios del negocio de Ferretería Cafetera, se identifican preliminarmente siete microservicios, cada uno con su propia base de datos, a los que se accede únicamente a través del API Gateway propio.

Microservicio	Responsabilidad
Usuario y autenticación	Usuarios, roles, inicio y cierre de sesión, emisión de JWT.
Catálogo	Productos, categorías, búsqueda y filtrado.
Proveedores	Registro de proveedores y asociación a productos.
Inventario	Movimientos, saldos por bodega, kardex y trazabilidad.
Compras y devoluciones	Órdenes de compra, recepción de mercancía, devoluciones.
Notificaciones y alertas	Alertas por stock bajo.
Reportes y dashboard	Existencias, valorización, rotación e indicadores.

9. Requerimientos Funcionales y No Funcionales.

Para los requisitos funcionales se utilizó el id REQ-F y para los no funcionales el id REQ-NF, con el objetivo de facilitar su búsqueda y su trazabilidad. Cada requisito tiene su propio id y todos los id son únicos.

La prioridad Esencial indica que, sin el requisito, el sistema no cumple su propósito; la prioridad Condicional indica que aporta valor, pero puede implementarse en una fase posterior si el tiempo lo exige. La prioridad ordena el trabajo, no descarta funcionalidad: varios requisitos del enunciado se consolidaron en un solo REQ-F cuando cubrían el mismo dominio.

- **Requerimientos Funcionales**

Modulo de Autenticación y usuarios

ID: REQ-F-001

Tipo: Funcional.

Prioridad: Esencial.

Origen: Control de acceso por rol.

Descripción: El sistema debe permitir a los usuarios iniciar y cerrar sesión con su usuario y contraseña, y recuperar o cambiar su contraseña. Al iniciar sesión, el sistema identifica al usuario con uno de los tres roles que son Administrador, Operador de bodega o Vendedor.
Justificación: Es la base para saber quién realiza cada movimiento y para diferenciar qué puede ver o hacer cada tipo de usuario.
Dependencias: Ninguna.
Riesgo: Contraseñas débiles o compartidas entre empleados facilitan accesos no autorizados y le quitan valor a la trazabilidad de los movimientos. Una recuperación sin verificación de identidad permite que un tercero tome control de una cuenta.

ID: REQ-F-002
Tipo: Funcional.
Prioridad: Esencial.
Origen: Control de acceso por rol
Descripción: El sistema debe permitir al Administrador registrar, consultar, actualizar y desactivar usuarios, y asignarles un rol.
Justificación: Los empleados de las tres sedes cambian con el tiempo, por lo que se necesita un mecanismo propio de alta, cambio de rol y baja, sin perder el historial de lo que cada usuario hizo.
Dependencias: REQ-F-001.
Riesgo: Asignar un rol equivocado da a un empleado acceso a funciones que no le corresponden, y borrar un usuario en lugar de desactivarlo dejaría movimientos sin responsable.

ID: REQ-F-003
Tipo: Funcional.
Prioridad: Esencial.
Origen: Control de acceso por rol - Principio de menor privilegio.
Descripción: El sistema debe permitir o negar cada operación según el rol del usuario autenticado, tanto en las pantallas como en las rutas de la API. El Administrador puede realizar todas las operaciones; el Operador de bodega puede registrar movimientos, traslados y recepciones y consultar el stock, el Vendedor puede consultar productos y disponibilidad y registrar salidas por venta.
Justificación: Garantiza que cada persona solo haga lo que su cargo requiere y le da a los demás módulos un punto de control centralizado para aplicar los permisos.
Dependencias: REQ-F-001, REQ-F-002.

Riesgo: Un endpoint sin protección permite saltarse la interfaz y ejecutar operaciones para las que el rol no tiene permiso.

Módulo de Proveedores

ID: REQ-F-004

Tipo: Funcional.

Prioridad: Esencial.

Origen: Diagnóstico del problema.

Descripción: El sistema debe permitir al Administrador registrar, consultar, actualizar y dar de baja proveedores (nombre o razón social, NIT, contacto y datos de entrega) y asociar cada proveedor a uno o varios productos.

Justificación: Ferretería Cafetera compra a varios proveedores y hoy no existe un registro único de ellos; centralizarlos permite saber a quién comprarle cuando un producto se agota. Dependencias: REQ-F-003.

Riesgo: Registrar al mismo proveedor dos veces con NIT o nombre distinto genera inconsistencias en las órdenes de compra, y dar de baja un proveedor con productos u órdenes abiertas deja referencias sin destino.

Modulo de Productos y categorías

ID: REQ-F-005

Tipo: Funcional.

Prioridad: Esencial.

Origen: Análisis de sistemas similares – Necesidad operativa de la ferretería.

Descripción: El sistema debe permitir al Administrador crear, consultar, actualizar y dar de baja productos, con código, nombre, descripción, categoría, unidad de medida, precio y proveedor asociado, y organizarlos en categorías y subcategorías. La baja de un producto debe ser lógica: deja de estar disponible, pero no se elimina físicamente.

Justificación: El catálogo es la base de los demás módulos, y conservar los productos dados de baja mantiene el historial de movimientos sin vacíos.

Dependencias: REQ-F-003, REQ-F-004.

Riesgo: Códigos duplicados o unidades de medida mal definidas (por ejemplo, vender por metro lo que se compró por rollo) generan descuadres de inventario, y dar de baja una categoría con productos asignados los deja sin clasificar.

ID: REQ-F-006

Tipo: Funcional.

Prioridad: Esencial.

Origen: Diagnóstico del problema actual de Ferretería Cafetera - Sobrecarga de consultas de disponibilidad en alta demanda.

Descripción: El sistema debe permitir buscar y filtrar productos por nombre, código, categoría o proveedor.

Justificación: Reduce el tiempo que un vendedor tarda en encontrar un producto y evita que tenga que preguntar por él a otras áreas.

Dependencias: REQ-F-005.

Riesgo: Con un catálogo grande, las búsquedas sin índices adecuados vuelven lenta la consulta más frecuente del sistema.

Módulo de Bodegas y control de Stock

ID: REQ-F-007

Tipo: Funcional.

Prioridad: Esencial.

Origen: Diagnóstico del problema actual.

Descripción: El sistema debe permitir al Administrador registrar, consultar, actualizar y desactivar las bodegas

Justificación: El stock se controla por ubicación física; sin bodegas registradas no es posible saber dónde está cada producto.

Dependencias: REQ-F-003.

Riesgo: Desactivar una bodega que aún tiene existencias dejaría producto sin ubicación visible.

ID: REQ-F-008

Tipo: Funcional.

Prioridad: Esencial.

Origen: Diagnóstico del problema actual - El inventario se actualiza una vez al mes y no se descuenta al vender.

Descripción: El sistema debe permitir registrar entradas, salidas y ajustes de inventario, y actualizar automáticamente el saldo de cada producto en cada bodega. Las salidas por venta, ya sea en el mostrador o por pedido mayorista, deben descontar el saldo de inmediato y rechazarse si no hay

existencias suficientes; los ajustes deben registrar su motivo. El registro solo afecta el inventario; la facturación queda fuera del alcance.
Justificación: Reemplaza el kardex manual que se actualiza una vez al mes y corrige que hoy el inventario no se descuente automáticamente cuando se vende un producto.
Dependencias: REQ-F-005, REQ-F-007
Riesgo: Dos ventas simultáneas del mismo producto en horas de alta demanda podrían dejar el saldo en negativo si no se controla la concurrencia, y los ajustes sin justificación pueden ocultar pérdidas.

ID: REQ-F-009
Tipo: Funcional.
Prioridad: Esencial.
Origen: Diagnóstico del problema actual - Las sedes se consultan y se prestan producto por llamada o mensaje
Descripción: El sistema debe permitir trasladar producto entre bodegas, registrándolo como una salida en la bodega de origen y una entrada en la de destino dentro de una misma operación.
Justificación: Hoy las sedes se prestan producto entre sí sin dejar registro; el traslado formal mantiene el saldo correcto en ambas bodegas.
Dependencias: REQ-F-008.
Riesgo: Un traslado que se registra a medias hace desaparecer productos del inventario.

ID: REQ-F-010
Tipo: Funcional.
Prioridad: Esencial.
Origen: Diagnóstico del problema actual - Verificar existencias en otras sedes por llamada telefónica o mensaje
Descripción: El sistema debe permitir consultar la disponibilidad de un producto por bodega y de forma consolidada en todas las sedes, en una sola consulta.
Justificación: Elimina las llamadas y mensajes entre sedes para saber si hay producto y reduce la sobrecarga de preguntas de los vendedores en alta demanda.
Dependencias: REQ-F-008.
Riesgo: Si la consulta muestra un saldo desactualizado, se le puede prometer al cliente un producto que ya no existe. Además, es la consulta que más carga recibirá.

ID: REQ-F-011

Tipo: Funcional.

Prioridad: Esencial.

Origen: Necesidad de trazabilidad del inventario

Descripción: El sistema debe registrar, por cada movimiento de inventario, quién lo hizo, cuándo, de qué tipo fue, sobre qué producto y bodega y por qué cantidad, y permitir consultar ese historial por producto. Estos registros no se pueden modificar ni eliminar.

Justificación: Da trazabilidad completa al inventario y permite explicar cualquier diferencia entre lo registrado y lo que se cuenta físicamente.

Dependencias: REQ-F-008.

Riesgo: Si el historial se puede alterar, pierde su valor para auditar y para rastrear errores.

Módulo de Compras y Devoluciones

ID: REQ-F-012

Tipo: Funcional.

Prioridad: Esencial.

Origen: Diagnóstico del problema actual - Las órdenes de compra se generan solo cuando el encargado nota que algo se agotó.

Descripción: El sistema debe permitir al Administrador generar órdenes de compra a un proveedor, indicando los productos, las cantidades y la bodega de destino, y consultar y actualizar su estado.

Justificación: Formalizar las órdenes y seguir su estado permite planear las compras, saber qué mercancía está en camino y evitar pedir dos veces lo mismo.

Dependencias: REQ-F-004, REQ-F-005, REQ-F-007.

Riesgo: Generar una orden con un proveedor que no surte el producto, o dejar estados sin actualizar, muestra como pendiente mercancía que ya llegó o pide lo que no se necesita.

ID: REQ-F-013

Tipo: Funcional.

Prioridad: Esencial.

Origen: Necesidad operativa de bodega.

Descripción: El sistema debe permitir al Operador de bodega recibir mercancía contra una orden de compra, de forma total o parcial, generando automáticamente las entradas de inventario correspondientes.

Justificación: Conecta las compras con el stock: al recibir la mercancía el saldo sube sin necesidad de digitar de nuevo los datos.

Dependencias: REQ-F-008, REQ-F-012.

Riesgo: Recibir dos veces la misma orden duplicaría el stock, y las recepciones parciales mal registradas dejan la orden en un estado incorrecto.

ID: REQ-F-014

Tipo: Funcional.

Prioridad: Condicional.

Origen: Necesidad operativa de la ferretería.

Descripción: El sistema debe permitir registrar devoluciones de clientes, reingresando el producto al stock cuando esté en buen estado, y devoluciones a proveedores, descontando la cantidad del stock y dejando la referencia de la orden de compra de origen.

Justificación: Las devoluciones y las garantías son parte normal de la operación de una ferretería y deben reflejarse en el inventario para que el saldo siga siendo real.

Dependencias: REQ-F-008, REQ-F-013.

Riesgo: Reingresar como disponible un producto defectuoso genera ventas de mercancía en mal estado, y una devolución a proveedor que no se descuenta mantiene stock que ya no está físicamente.

Módulo de Alertas

ID: REQ-F-015

Tipo: Funcional.

Prioridad: Esencial.

Origen: Diagnóstico del problema actual - El encargado de turno detecta la escasez de forma manual

Descripción: El sistema debe permitir al Administrador configurar el umbral mínimo de stock de cada producto en cada bodega, y generar una alerta cuando el saldo baje de ese umbral, mostrarla en la interfaz y notificarla por correo u otro medio al Administrador y al Operador de la bodega afectada.

Justificación: Reemplaza la detección manual y permite reabastecer antes de que el producto se agote; cada producto se agota a un ritmo distinto, por eso el umbral es configurable.

Dependencias: REQ-F-005, REQ-F-007, REQ-F-008.

Riesgo: Umbrales mal calibrados generan avisos innecesarios o dejan pasar quiebres reales; repetir la alerta con cada movimiento acostumbra a los usuarios a ignorarla, y si el servicio de notificaciones falla, la alerta no llega.

ID: REQ-F-016

Tipo: Funcional.

Prioridad: Condicional.

Origen: Notas del equipo.

Descripción: El sistema debe sugerir cuándo y cuánto reabastecer un producto considerando su rotación histórica y la temporada, además del umbral fijo. La sugerencia solo se muestra al usuario; no genera órdenes de compra por sí sola.

Justificación: Un umbral fijo no refleja que en una ferretería la demanda aumenta con las obras y en fechas como fin de año; la sugerencia ayuda a anticiparse.

Dependencias: REQ-F-011, REQ-F-015.

Riesgo: Con poco historial al inicio, las sugerencias pueden ser poco precisas y generar exceso o falta de producto.

Módulo de Reportes y Dashboard

ID: REQ-F-017

Tipo: Funcional.

Prioridad: Condicional.

Origen: Necesidad operativa de la administración.

Descripción: El sistema debe permitir al Administrador generar reportes de existencias actuales por producto y bodega, de valorización del inventario y de los productos de mayor y menor rotación en un periodo.

Justificación: Da a la administración una visión unificada del inventario de las tres sedes, que hoy está dispersa en documentos manuales, y ayuda a decidir qué comprar y qué dejar de comprar.

Dependencias: REQ-F-005, REQ-F-008, REQ-F-011.

Riesgo: Las consultas pesadas sobre muchos movimientos pueden hacer lento el sistema si no se optimizan, y comparar periodos con eventos atípicos puede llevar a conclusiones equivocadas.

ID: REQ-F-018

Tipo: Funcional.

Prioridad: Condicional.

Origen: Notas del equipo.

Descripción: El sistema debe mostrar al Administrador un panel con indicadores clave: rotación, quiebres de stock y valor total del inventario.

Justificación: Permite ver el estado del negocio de un vistazo, sin tener que generar reportes uno por uno.

Dependencias: REQ-F-015, REQ-F-017.

Riesgo: Indicadores calculados con datos desactualizados llevan a decisiones erróneas.

- **Requerimientos No Funcionales**

ID: REQ-NF-001

Tipo: No Funcional.

Prioridad: Esencial.

Origen: Decisión de arquitectura del equipo

Descripción: El sistema debe construirse como un conjunto de servicios independientes y débilmente acoplados, cada uno responsable de un dominio del negocio y con su propia base de datos. Los servicios se comunican solo mediante su API REST o mensajes, sin que ninguno acceda directamente a la base de datos de otro.

Criterio de aceptación: Dado que se revisan el repositorio y el despliegue, cuando se inspecciona cada servicio, entonces cada uno debe poder construirse y desplegarse por separado, y ninguno debe tener credenciales ni acceso a la base de datos de otro.

Justificación: Es la decisión arquitectónica central del proyecto: permite escalar y actualizar cada dominio de forma independiente y evita que un cambio en los datos de un servicio rompa a los demás.

Dependencias: Ninguna.

Riesgo: Servicios que comparten código o se llaman en cadena de forma excesiva se convierten en un monolito distribuido, con la complejidad de los microservicios pero sin sus ventajas. Además, las consultas que cruzan servicios se vuelven más complejas.

ID: REQ-NF-002

Tipo: No Funcional.

Prioridad: Esencial.

Origen: Decisión de arquitectura del equipo.

Descripción: Todas las peticiones externas dirigidas al sistema deben pasar por un API Gateway desarrollado por el equipo, que enruta cada petición al microservicio correspondiente y puede centralizar la autenticación. No se usarán soluciones de terceros como Kong.

Criterio de aceptación: Dado que un cliente externo intenta comunicarse directamente con un microservicio sin pasar por el Gateway, cuando envía la petición, entonces no debe ser posible

acceder al servicio; y cuando la petición pasa por el Gateway, entonces debe enrutarse al servicio correcto.

Justificación: Centraliza la seguridad en un único punto de entrada y simplifica el acceso de los clientes, sin depender de una solución comercial cerrada.

Dependencias: REQ-NF-001, REQ-F-003.

Riesgo: Si el Gateway falla, todo el sistema queda inaccesible; por eso debe poder replicarse.

ID: REQ-NF-003

Tipo: No Funcional.

Prioridad: Esencial.

Origen: Restricción tecnológica del entorno académico.

Descripción: Cada microservicio y sus dependencias como bases de datos, mensajería deben ejecutarse en contenedores Docker, con una imagen propia por servicio, y el sistema debe desplegarse y gestionarse con Kubernetes. Las diferencias de configuración entre entornos se manejan con variables de entorno, sin modificar el código. Docker Compose solo puede usarse como apoyo en el desarrollo local.

Criterio de aceptación: Dado que se cuenta con las imágenes y los manifiestos del proyecto, cuando se aplican sobre un clúster de Kubernetes distinto al de desarrollo ajustando solo las variables de entorno, entonces todos los servicios deben quedar en ejecución y accesibles a través del Gateway.

Justificación: Garantiza que el sistema se comporte igual en desarrollo, pruebas y despliegue, y permite administrar, reiniciar y escalar los servicios de forma automatizada.

Dependencias: REQ-NF-001.

Riesgo: La curva de aprendizaje de Kubernetes puede retrasar el despliegue, y las direcciones, contraseñas o rutas escritas en el código impiden trasladar el sistema a otro entorno.

ID: REQ-NF-004

Tipo: No Funcional.

Prioridad: Esencial.

Origen: Análisis de demanda en temporadas altas

Descripción: El sistema debe permitir aumentar el número de réplicas de un microservicio, por ejemplo el de inventario que atiende las consultas de stock, sin modificar ni volver a desplegar los demás.

Criterio de aceptación: Dado que aumenta la demanda sobre las consultas de stock, cuando se incrementan las réplicas del servicio de inventario, entonces el sistema debe repartir la carga entre ellas sin afectar a los demás servicios.
Justificación: En temporadas de alta demanda las consultas de stock crecen mucho más que el resto de operaciones, y escalar solo esa parte ahorra recursos.
Dependencias: REQ-NF-003.
Riesgo: Un servicio que guarda estado en su memoria no se puede replicar correctamente; el estado debe residir en la base de datos.

ID: REQ-NF-005

Tipo: No Funcional.
Prioridad: Esencial.
Origen: Análisis de experiencia de usuario en el mostrador
Descripción: El sistema debe responder al 95% de las consultas de stock y de productos en menos de 2 segundos con una carga de 50 usuarios concurrentes, valor definido por el equipo y ajustable según el plan de pruebas.
Criterio de aceptación: Dado un escenario de prueba con 50 usuarios simulados consultando el stock, cuando se ejecutan las pruebas de carga, entonces al menos el 95% de las respuestas deben completarse en menos de 2 segundos.
Justificación: La consulta de disponibilidad es la más frecuente, y un tiempo de respuesta alto afecta directamente la atención al cliente.
Dependencias: REQ-F-006, REQ-F-010.
Riesgo: Las consultas sin índices, o las llamadas en cadena entre servicios, aumentan la latencia a medida que crece el catálogo.

ID: REQ-NF-006

Tipo: No Funcional.
Prioridad: Esencial.
Origen: Buenas prácticas de seguridad en autenticación.
Descripción: La autenticación debe usar un mecanismo estándar, las contraseñas deben almacenarse cifradas con un algoritmo de hash seguro, sin guardarlas nunca en texto plano, y las comunicaciones deben protegerse.

Criterio de aceptación: Dado que un usuario se registra o cambia su contraseña, cuando el dato se guarda en la base de datos, entonces debe quedar almacenado únicamente el hash; y dado que se invoca un servicio sin un token válido, entonces la petición debe ser rechazada.
Justificación: Protege las credenciales incluso ante un acceso indebido a la base de datos e impide operaciones de usuarios no autenticados.
Dependencias: REQ-F-001.
Riesgo: Un algoritmo de hash débil, o tokens que nunca vencen, facilitan ataques de fuerza bruta y la reutilización de sesiones robadas.

ID: REQ-NF-007

Tipo: No Funcional.
Prioridad: Esencial.
Origen: Ley 1581 de 2012 (Protección de Datos Personales, Colombia).
Descripción: El sistema debe tratar los datos personales que almacena, empleados, contactos de proveedores y de clientes mayoristas. Conforme a la Ley 1581 de 2012: recolectar solo los datos necesarios, usarlos únicamente para la gestión del inventario y restringir su consulta a los roles autorizados. Los usuarios y los movimientos no se eliminan físicamente; se desactivan para conservar el historial.
Criterio de aceptación: Dado que se revisan los datos personales almacenados, cuando se comparan con los que el sistema necesita, entonces solo deben existir los datos necesarios y su consulta debe estar limitada por rol; y dado que se desactiva un usuario, entonces su historial de movimientos debe permanecer.
Justificación: Es una obligación legal para cualquier sistema que trate datos personales en Colombia, y conservar el historial evita perder la trazabilidad de quién hizo cada movimiento.
Dependencias: REQ-F-002, REQ-F-003.
Riesgo: El incumplimiento normativo puede derivar en sanciones, y guardar datos que no se necesitan aumenta el impacto de una filtración.

ID: REQ-NF-008

Tipo: No Funcional.
Prioridad: Esencial.
Origen: Análisis de continuidad ante fallos.

Descripción: Los datos de cada microservicio deben conservarse en volúmenes persistentes, de modo que no se pierdan cuando un contenedor se reinicia, y las bases de datos deben contar con copias de seguridad periódicas que se puedan restaurar.

Criterio de aceptación: Dado que un contenedor de base de datos se reinicia o se recrea, cuando el servicio vuelve a iniciar, entonces los datos registrados antes del reinicio deben seguir disponibles; y dado que se ejecuta el proceso de respaldo, entonces debe generarse una copia que pueda restaurarse.

Justificación: El inventario es información crítica del negocio; perderla obligaría a recontar físicamente las tres bodegas.

Dependencias: REQ-NF-003.

Riesgo: Las copias de seguridad que nunca se prueban pueden fallar justo cuando se necesitan.

ID: REQ-NF-009

Tipo: No Funcional.

Prioridad: Esencial.

Origen: Necesidad de diagnóstico en sistemas distribuidos.

Descripción: Cada microservicio debe generar registros (logs) consultables de su actividad y de sus errores, con un identificador que permita seguir una misma petición entre servicios; de manera deseable, el sistema debe exponer métricas básicas de monitoreo.

Criterio de aceptación: Dado que ocurre un error en una petición, cuando se revisan los registros del servicio, entonces debe encontrarse el error con su fecha y hora y con el identificador que permita seguir la petición entre servicios.

Justificación: En un sistema distribuido, encontrar la causa de una falla sin registros es muy difícil.

Dependencias: REQ-NF-001.

Riesgo: Los registros dispersos en cada contenedor, o que incluyan datos sensibles como contraseñas, dificultan el diagnóstico y comprometen la seguridad.

ID: REQ-NF-010

Tipo: No Funcional.

Prioridad: Esencial.

Origen: Buenas prácticas de microservicios.

Descripción: Cada microservicio debe exponer la documentación de su API en formato OpenAPI/Swagger.

Criterio de aceptación: Dado que un servicio está en ejecución, cuando se accede a su documentación, entonces deben poder consultarse todos sus endpoints, parámetros y respuestas.
Justificación: Los contratos de API son el acuerdo entre los servicios y entre los integrantes del equipo; sin documentación, un cambio puede romper a otro servicio sin que nadie lo note.
Dependencias: REQ-NF-001.
Riesgo: Una documentación que no se actualiza junto con el código termina describiendo una API distinta a la real.

10. Posibles ideas y funcionalidades futuras e innovadoras

- Alertas de stock predictivas basadas en la rotación histórica, no solo en un umbral fijo de bajo stock.
- Trazabilidad visual tipo línea de tiempo del producto.
- Notificaciones multicanal nativas desde el diseño y no como integración externa.
- Dashboard que sugiere acciones, no solo muestra números.
- API Gateway propio con observabilidad desde el diseño como diferencia frente al vendedor lock-in de las soluciones comerciales.

11. Referencias

- Conventional Commits: https://www.conventionalcommits.org/
- ERPNext: https://github.com/frappe/erpnext
- SAP Business One: https://erpresearch.com/en-us/sap-business-one-inventory-management-module
- Siigo: https://www.siigo.com/gestion-de-inventario/
- Odoo: https://www.odoo.com/app/inventory